
IMAGE STEGANOGRAPHY

PROJECT REPORT

Submitted to

Dr. Altaf Badar

Assistant Prof

Dept of Electrical Engineering

Submitted by

CH. Raja Vignesh

22EEB0A33

B.Tech II Year

EEE - A



NATIONAL INSTITUTE OF TECHNOLOGY, WARANGAL

Project Overview:

Image Steganography project explores the art of concealing information within digital images. In an era where secure communication is paramount, this project aims to implement and demonstrate image steganography using python.

Project Objectives:

➤ **Friendly User Interface:**

Develop an intuitive and user-friendly interface that allows users to easily encrypt data in image and get hidden information from an image. In this project the interface is developed using Tkinter which is a python library which is used to construct graphical user interface applications

➤ **Encoding Functionality**

- Enable users to select an image file for hiding their secret message.
- Implement a method to input the secret message.
- Utilize steganographic techniques to embed the message within the chosen image, considering factors like image size, quality, and potential distortions.

➤ **Decoding Functionality**

- Allow users to choose an image file that may contain a hidden message.
- Implement the decoding process to extract and display the hidden message from the selected image.

➤ **Organize Code and Comments**

- the code in a modular and readable manner.
- Include comments within the code to make it understandable for future developers or users.

INTRODUCTION

Image Steganography: Image Steganography is the process of hiding information which can be text, image or video inside a cover image. The secret information is hidden in a way that it is not

visible to the human eyes. In this project steganography is performed using Least Significant Bit (LSB) substitution method.

APPLICATIONS OF IMAGE STEGANOGRAPHY

- **Service Communication:** Steganography can be used to hide confidential information within images, providing a secure means of communication.
- **Digital Watermarking:** Steganography can be employed to embed invisible watermarks in images, helping to protect intellectual property by associating ownership information with the media.
Steganography can be used to embed hidden instructions or additional content within educational multimedia material.
- **Document Authentication:** For sensitive documents, steganography can be employed to detect tampering by embedding information that indicates the authenticity of the document.
- **Integrity Verification:** Embedding information in images can serve as a means of verifying the integrity of the data. If the image is altered, the hidden information may be compromised.
- **Biometric Data Embedding:** Steganography can be applied to embed biometric data within images for authentication purposes.
- **Privacy Protection:** Users may use steganography to hide personal or sensitive information within images before sharing them on social media platforms.
- **Undercover Operations:** In the field of law enforcement or military operations, steganography can be used for covert communication, allowing agents to transmit information discreetly.

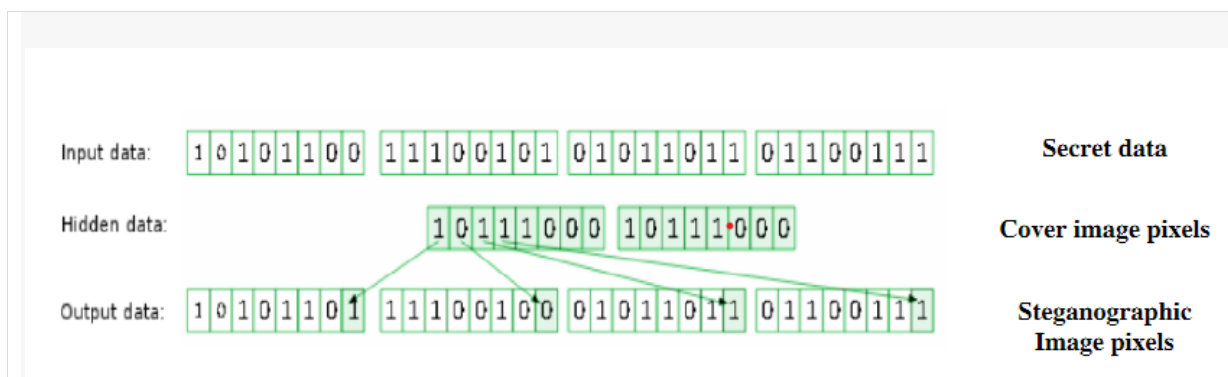
TECHNICAL DISCUSSION

Least Significant Bit (LSB) substitution method is used to hide data in the image. In this approach, this code essentially hides information in the least significant bits of the RGB values of the image.

Since every image contains three components (RGB). This pixel data is encoded and saved in one byte. The initial bits of each pixel's information can be adjusted to store the hidden text. The preliminary condition for this is that the text to be stored be smaller or equal in size to the image used to hide the information.

The MSB (most significant bits) of the concealed message image are saved in the LSB (least significant bits) of the cover image. It is well understood that pixels in an image are stored as bits. The intensity of each pixel in a grayscale image is represented in 8 bits (1byte). Similarly, each pixel in a colour (RGB-red, green, blue) image requires 24 bits (8 bits for each layer).

When the LSB bit is changed, the human visual system (HVS) cannot detect changes in the colour or intensity of a pixel. This is psycho-visual redundancy because it can be used to store information in these bits while noticing no significant difference in the image.



IMPLEMENTATION AND RESULTS

This project operates as a menu driven program and interacts with the user by making use of Graphical User Interface (GUI). When the user makes a choice to encode data into an image or decode data from an image the chosen operation is performed by respective functions.

The process of encoding data in an image can be represented by following pseudocode:

```

Function encode():
    Prompt the user to select an image file
    Read the image using OpenCV
    Get the size of the image
    Call the supply function to get the message from the user
    Check if the message is empty
    Prompt the user to select a destination for the encoded image
    Encode the data in the image using hidedata function
    Save the encoded image
    Resize the encoded image to the original size
    Save the resized image with optimized quality

```

Here the hidedata function is used to perform the encryption

```

Function hidedata(img, data):
    Append "$$" to the data (secret key)
    Initialize d_index to 0
    Convert data to binary using data2binary function
    Get the length of the binary data (len_data)
    For each value in img:
        For each pixel in value:
            Convert RGB values of the pixel to binary
            If d_index is less than len_data:
                Modify the least significant bit of the red
component
                Increment d_index
            If d_index is less than len_data:
                Modify the least significant bit of the green
component
                Increment d_index
            If d_index is less than len_data:
                Modify the least significant bit of the blue
component
                Increment d_index
            If d_index is greater than or equal to len_data:
                Break the loop
    Return the modified image

```

Similarly, decryption of data is done by using the function find_data. The function is expressed by following pseudocode

```

Function find_data(img):
    Initialize an empty string bin_data
    For each value in img:
        For each pixel in value:
            Convert RGB values of the pixel to binary

```

```
        Append the least significant bits of RGB to bin_data
    Split bin_data into 8-bit chunks (all_bytes)
    Convert each chunk to a character and concatenate them into
readable_data
    If the last two characters of readable_data are "$$":
        Break the loop
    Return readable_data excluding the last two characters
```

The decode function is expressed by following pseudocode

```
Function decode():
    Create a tkinter window
    Prompt the user to select an image file
    Destroy the tkinter window
    Read the image using OpenCV
    Call the find_data function to decode the message
    Display the decoded message in a new tkinter window
```

The program is executed by using python programming language and the libraries used in the program are:

OpenCV: OpenCV is the huge open-source library for computer vision, machine learning, and image processing. It can process images and videos to identify objects, faces, or even the handwriting of a human. When it integrated with various libraries, such as NumPy, python is capable of processing the OpenCV array structure for analysis.

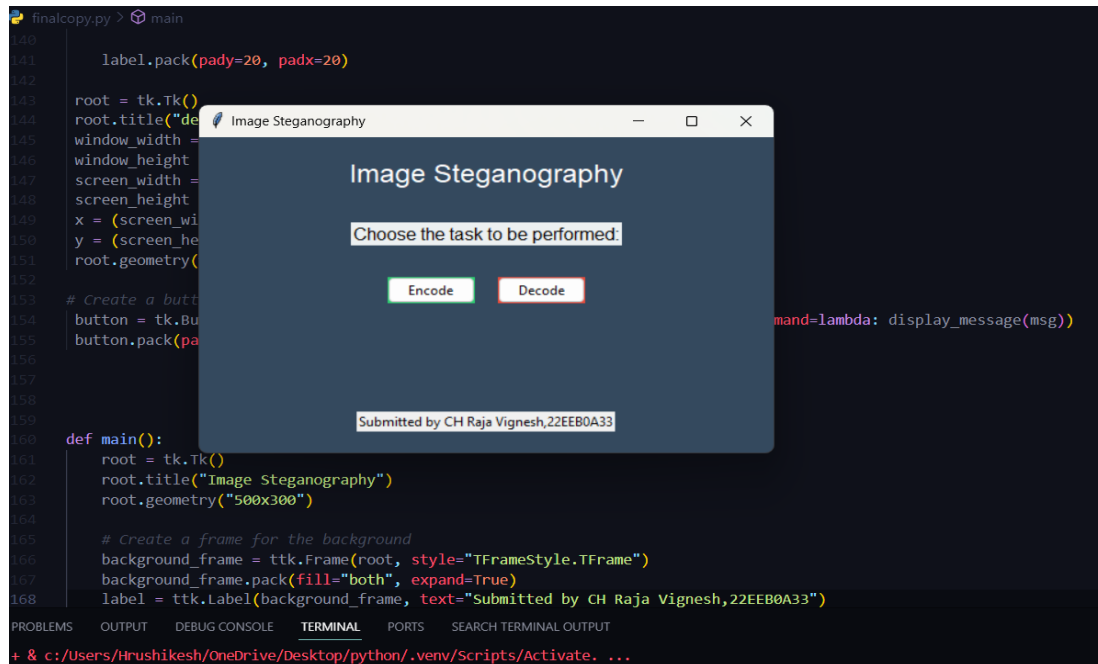
NumPy: NumPy is a library for numerical operations in Python. It is used here for handling arrays, which are often used in image processing. Here, NumPy is used so that it could even take NumPy arrays as an input in data2binary function.

Image from PIL library: Python Imaging Library is a free library for the Python programming language that adds support for opening, manipulating, and saving many different image files. The `Image` class from the Python Imaging Library (PIL) is used for opening and resizing images.

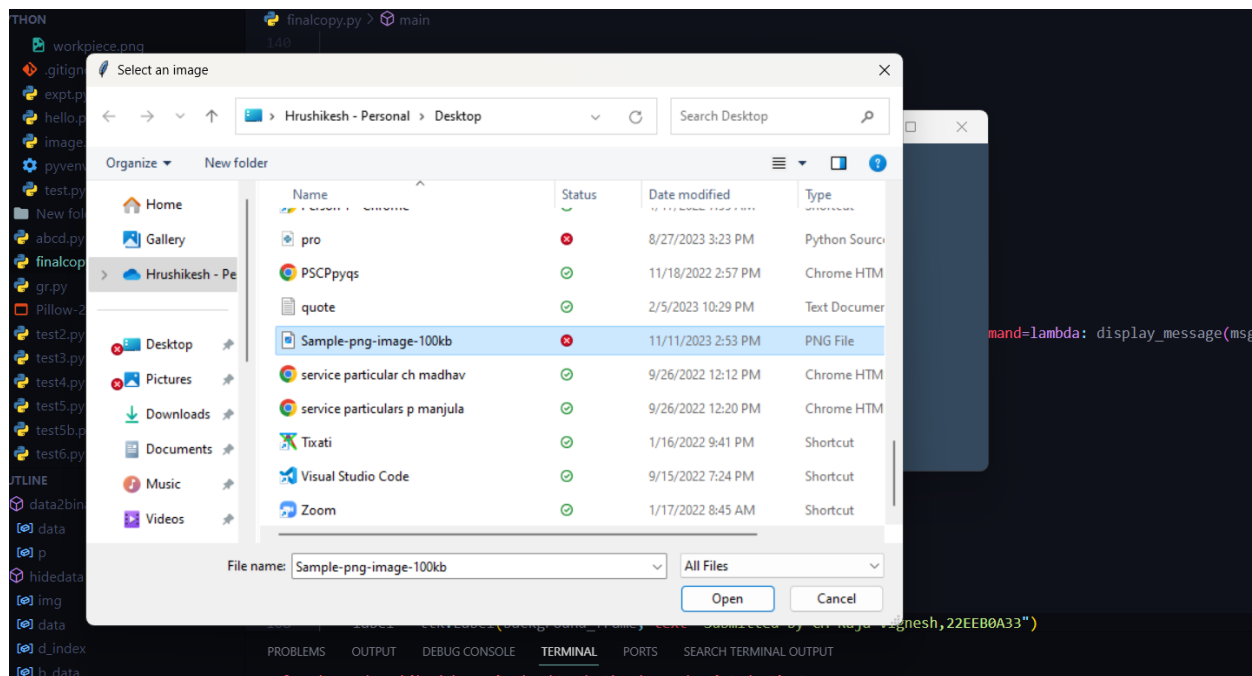
Tkinter: It is used for creating the graphical user interface (GUI) of the application. `FileDialog` module provides classes and factory functions for creating file/directory selection windows. `Tkinter messagebox` is a module in python which provides a different set of dialogues that are used to display message boxes. The `ttk` module provides access to the Tk themed widget set.

The program's outcome is as follows:

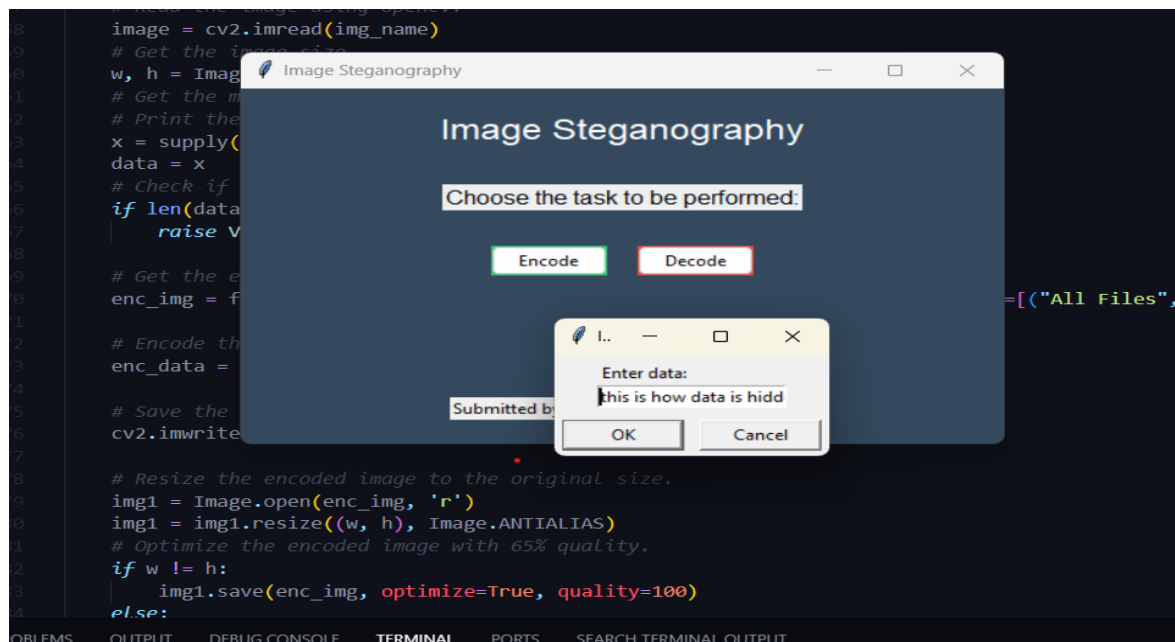
The main window gets displayed



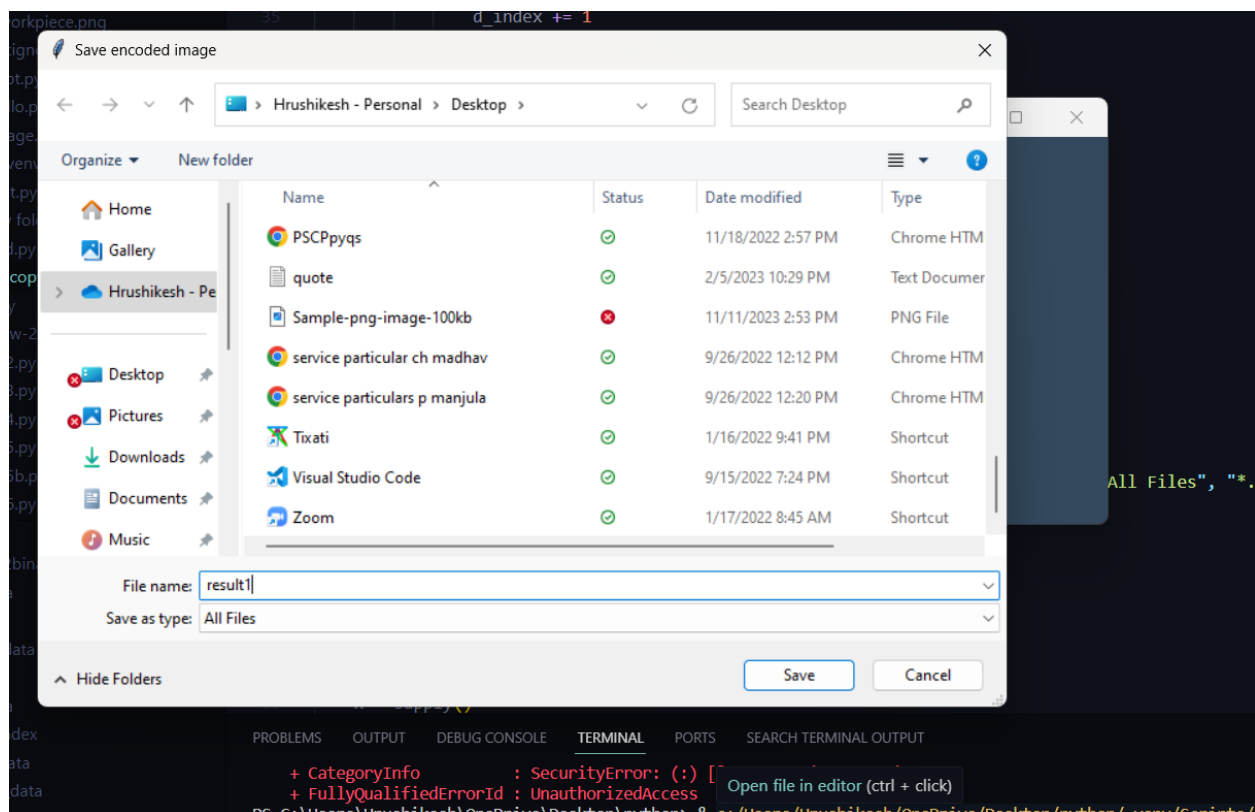
If we choose to encode data in an image we click encode and we need to choose the file to be encoded. A file can be chosen with the help of a File dialog which is opened upon clicking the encode button



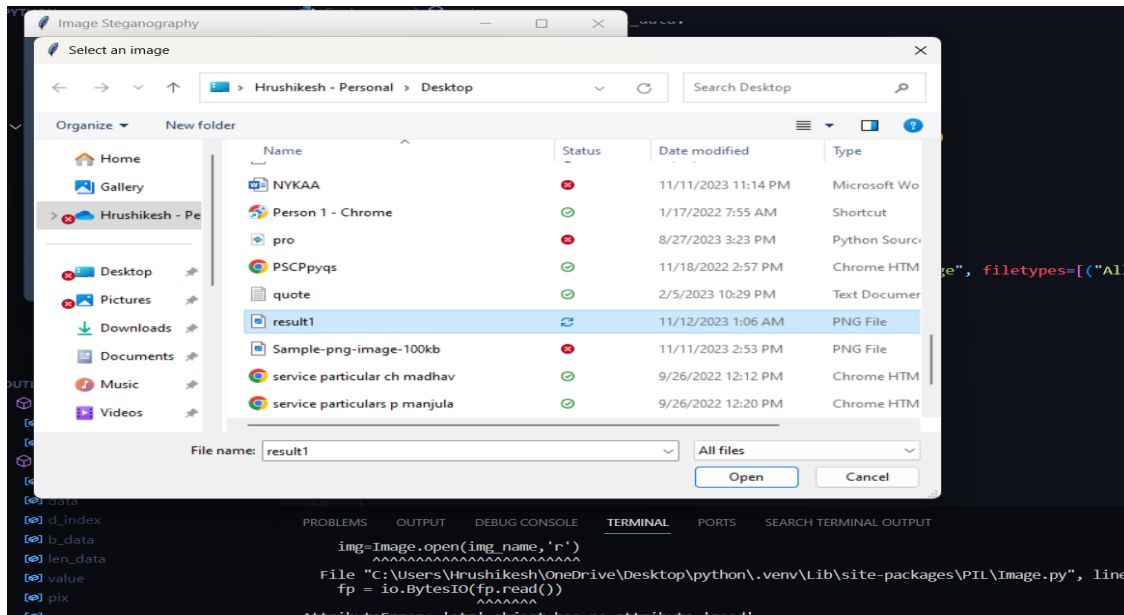
Then enter the data to be hidden in the dialog box and click on OK.



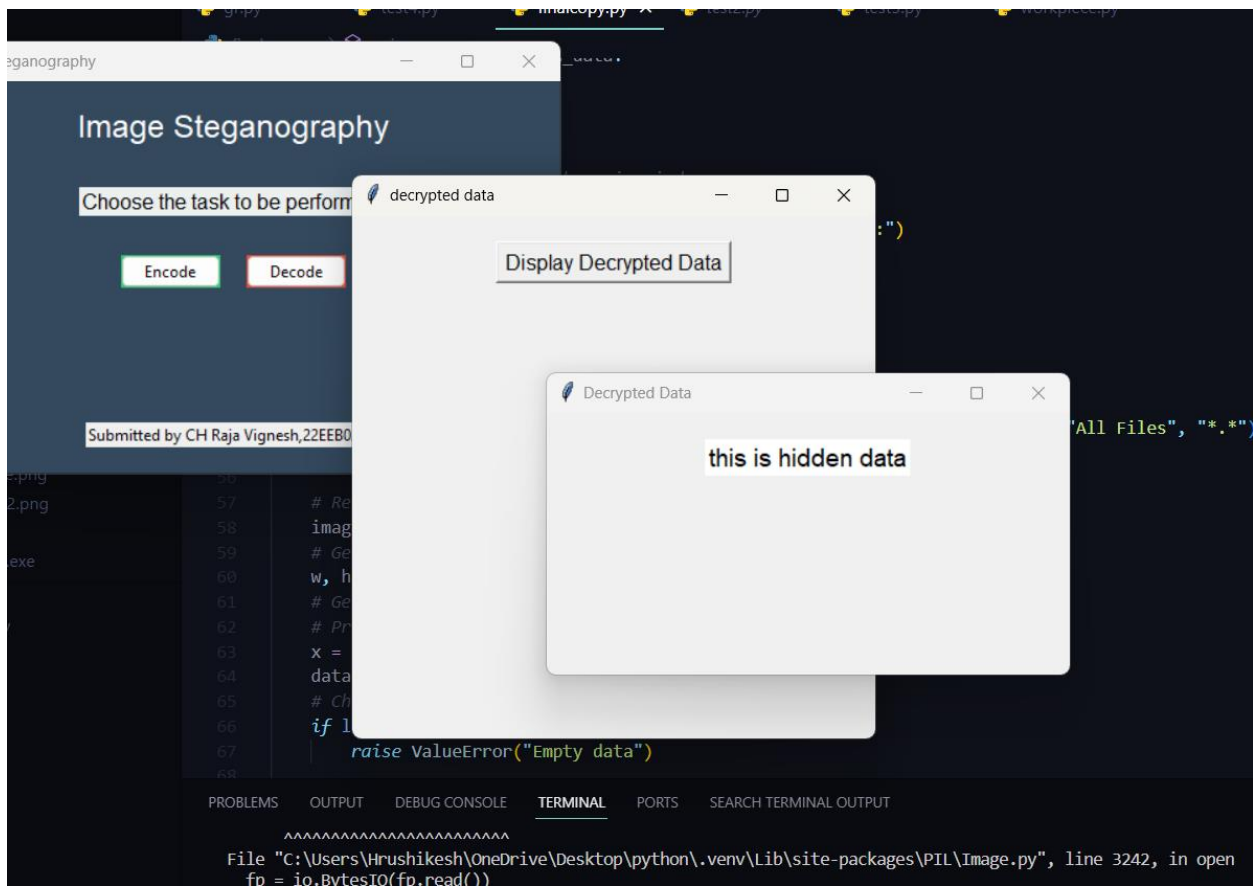
Enter the name of new image which has encoded image and save in desired location. After this the image is saved in the location.



Now to decode the image click on decode and select the file.



Now click on display decrypted data to see the data.



So, the data is successfully encoded and decoded. The initial(right) and final(left) images are:

